

Alohomora

Unlocking Magical Bits in Python

Prashant Sinha
University of Delhi

  @prashnts

<https://noop.pw>
ps+xor@noop.pw

18-03-2017



#Who?

- Math and IT Major at *University of Delhi*
- Software Developer at *Agora NHS*
- Proudly a Pythonista
- Loves blueberry ice creams

#Agenda

- Decorators
- Monkey Patching
- Metaclasses

#Wizarding Basics

What every young witches and wizards should know about Python?

aka Object Oriented Programming Concepts

#Wizarding Basics

Everything is an object.

Objects can:

- be assigned to a variable
- be copied
- be passed as arguments to functions
- hold attributes

Functions are first-class citizens.

You can treat function just like any other variable. They can be passed as arguments to other functions.

#Decorators

High-order functions

aka Functions that operate on other functions

#Decorators

```
def energy_shield(func):  
    def inner(*args, **kwargs):  
        try:  
            return func(*args, **kwargs)  
        except Exception as e:  
            print(f"Contained exception {e}")  
    return inner
```

```
@energy_shield
```

```
def very_fragile_planet(args):  
    if random.random() > 0.9:  
        raise PlanetExplosion
```

```
very_fragile_planet = energy_shield(very_fragile_planet)
```

#Decorators

A decorator is a *function* which accepts a *function* and returns **another** *function*.

#Decorators

Decorators can be stacked.

```
@with_boosting_rocket  
@with_artificial_gravity(g=9.8)  
@with_climate_control  
def room( ... ):  
    ...
```

```
room = with_boosting_rocket(with_artificial_gravity(g=9.8)(with_climate_control(room)))
```

#Decorators

Order matters.

```
@foo
@bar
def f( ... ):
    pass
```

f = foo(bar(f))

```
@bar
@foo
def f( ... ):
    pass
```

f = bar(foo(f))

#Decorator Patterns

Using Decorators

#Decorator Patterns

Debug logging

```
def sad_func(*args):  
    print("Input args:", args)  
    out = do_something(*args)  
    print("Output is:", out)  
    return out
```

#Decorator Patterns

Debug logging

```
def sad_func(*args):  
    print("Input args:", args)  
    out = do_something(*args)  
    print("Output is:", out)  
    return out  
  
def sadder_func(*args):  
    print("Input args:", args)  
    out = do_something_again(*args)  
    print("Output is:", out)  
    return out  
  
def saddest_func(*args):  
    print("Input args:", args)  
    out = do_something_yet_again(*args)  
    print("Output is:", out)  
    return out
```

That's a lot of repeated code!
We can do better.

#Decorator Patterns

Debug logging

```
def debug_tap(func):
    fn_name = func.__name__
    def inner(*args, **kwargs):
        out = func(*args, **kwargs)
        print(f"{fn_name}")
        print(f"→ args = {pprint(args)}")
        print(f"→ kwargs = {pprint(kwargs)}")
        print(f"⇒ returnvalue = {pprint(out)}")
        return out
    return inner
```

```
@debug_tap
def sad_func(*args):
    return do_something(*args)
```

```
@debug_tap
def sadder_func(*args):
    return do_something_again(*args)
```

```
@debug_tap
def saddest_func(*args):
    return do_something_yet_again(*args)
```

#Decorator Patterns

Decorators with parameters

aka Dynamically defined decorators

```
@truncate(count=30)
def random_string():
    ...
    return very_long_string
```

```
def truncate(count):
    def decorator(func):
        def replacement(*args, **kwargs):
            out = func(*args, **kwargs)
            return out[:count]
        return replacement
    return decorator
```

#Decorator Patterns

Auto-retry network I/O

```
def retry(max_tries):
    def decorator(func):
        def inner(*args, **kwargs):
            t = kwargs.pop('tries', 1)
            try:
                return func(*args, **kwargs)
            except OSError:
                if t ≤ max_tries:
                    return inner(tries=t + 1, *args, **kwargs)
                raise
        return inner
    return decorator
```

```
@retry(max_tries=3)
def get(url):
    return requests.get(url)
```

#Monkey Patching

Altering code at Runtime

#Monkey Patching

Everything is an object.
Hence everything is in the memory.

#Monkey Patching

To be useful, each of these objects must be *referenced* by a variable or be in a module.

#Monkey Patching

If we change those *references* to point at something of our own, we've created a patch.

#Monkey Patching

By patching, we can dynamically modify a class or a module during runtime.

#Monkey Patching

In Python, the `unittest.mock` module provides a helper for patching *stuff*.

#Patching Patterns

Why though?

#Patching Patterns

Add functionality

```
import requests

req_original = requests.api.request
_cache = {}

def req_cached(method, url, **kwa):
    request_hash = f'{method} {url}'
    try:
        return _cache[request_hash]
    except KeyError:
        response = req_original(method, url, **kwa)
        _cache[request_hash] = response
    return response
```

```
from unittest.mock import patch

@patch('requests.api.request', req_cached)
def foo(*a, **kwa):
    requests.get('http://foo.bar')
    requests.get('http://foo.bar')
```

#Patching Patterns

Inject workarounds

```
responses = {
    '/foo': <Response 200>,
    '/foo/bar': <Response 200>,
    '/baz': <Response 202>,
}

def known_responses(method, url, **kwa):
    try:
        return responses[url]
    except KeyError:
        return <Response 404>

@patch('requests.api.request', known_responses)
def test_response_parser():
    ...
```

#Patching Patterns

Police Coworkers

```
def eval_nope(*args, **kwargs):  
    raise DoNotUseEvalError  
  
__builtin__.eval = eval_nope
```

#Metaclasses

Class of the Class

#Metaclasses

If everything is an object, what created them?

In Python

- Integers are objects of type `int`.
- Strings are objects of type `str`.
- Functions are objects of type `function`.
- Classes are object of type, well, `type`.

#Metaclasses

A metaclass in Python creates a class.

#Metaclasses

`type` is a metaclass which Python uses to create all the classes.

#Metaclasses

`type(object)` → the object's type
`type(name, bases, dict)` → a new type

#Metaclasses

`type` is an instance of class `type`.

#Metaclasses

Dogeified Classes

```
class dogetype(type):  
    def __new__(cls, name, bases, attrs):  
        attrs_wow = {"so_" + x + "_wow": y for x, y in attrs.items()}  
        return type(name, bases, attrs_wow)
```

```
class NotWow(metaclass=dogetype):  
    foo = 1
```

>

```
class NotWow:  
    so_foo_wow = 1
```

#Metaclasses

If we subclass `type`, then we gain control over how the *classes are defined*.

#Metaclasses

A metaclass gives you the super power of customising how the class attributes should behave, add extra functionalities, or enforce standards.

#Takeaways

How to be a responsible wizard?

#Takeaways

Explicit is better than implicit.

Don't go around patching everything. It may make it super hard for other people to understand your code.

#Takeaways

Simple is better than complex.

Don't use metaclasses if extending a base class is possible.

#Takeaways

Complex is better than complicated.

Do use metaclasses if class attributes are dynamically added.

#Takeaways

Readability counts.

Don't add too many decorators to a function.

#Takeaways

Be adventurous!

#Questions?

#References

This presentation uses Creative Commons CC-BY-SA-4.0 License.

Content in this presentation were referenced from a lot of sources. The significantly important ones are given here:

- <http://www.onthelambda.com/2016/07/08/using-python-decorators-to-be-a-lazy-programmer-a-case-study/>
- <https://web.archive.org/web/20090106184755/http://scratch.tplus1.com/decoratortalk/slides.html>
- <https://www.youtube.com/watch?v=ZrUIRSVv1gw>
- <https://blog.codinghorror.com/monkeypatching-for-humans/>
- <http://stackoverflow.com/a/6581949/2648954>